

Introduzione

Il progetto The Cycle Game è un videogioco strutturato seguendo metodologie di sviluppo agile, in particolare attraverso l'utilizzo delle User Stories.

Le User Stories descrivono le funzionalità dal punto di vista dell'utente finale e guidano le scelte architetture e progettuali. Ogni User Story è collegata a specifici componenti del sistema, consentendo di tracciare in modo diretto l'implementazione dei requisiti.

Tecnologie utilizzate

Il progetto è stato implementato nella versione 21 del linguaggio, perché presenta sia feature moderne ed ergonomiche sia un'elevata stabilità e portabilità, essendo comunque una versione LTS.

La gestione delle dipendenze e della compilazione è affidata a Maven, consentendo l'automatizzazione del download delle librerie necessarie e della gestione delle versioni corrette di ciascuna dipendenza, migliorando la riproducibilità e la manutenibilità del progetto. Per l'ambiente grafico e la gestione dell'input da tastiera e mouse, si è scelto di usare JavaFX, data la sua flessibilità, semplicità e integrazione diretta con Java.

L'apparato di testing è implementato usando JUnit per la gestione dei test veri e propri. Per gestire componenti complesse o dipendenze non rilevanti ai fini del test, in particolare le view basate su JavaFX, è stato utilizzato Mockito. Questo approccio permette di creare mock delle classi, evitando l'esecuzione diretta del codice grafico nelle classi di test e garantendo un test efficace della logica di gioco isolata dal layer grafico. Infine, per quanto riguarda il caricamento dei dati di gioco e la gestione dei salvataggi, si è scelto di utilizzare Jackson, in quanto scelta standard in ambito industriale per la serializzazione e deserializzazione di file JSON in Java, garantendo efficienza e affidabilità nella lettura e scrittura dei dati persistenti.

Architettura del gioco

L'architettura del gioco è tipo **MVC**, Model-View-Controller, che permette di separare i dati e la logica del flusso del gioco (Model) da ciò che viene visualizzato a schermo come output delle azioni dei giocatori (View) tramite dei gestori che scelgono cosa mostrare e quando (Controller). I controller permettono quindi la comunicazione da Model a View, ma anche da View a Model, interrogando ed eventualmente modificando i dati di quest'ultimo sulla base degli input dati dai giocatori nella View.

Ogni classe java del gioco rientra quindi in una di queste 3 categorie.

Per garantire una maggiore modularità e costruire una gerarchia tra classi abbiamo costruito un controller dedicato per ogni view, garantendo un accoppiamento forte tra View e Controller. Questo approccio permette di mantenere il codice organizzato e facilmente estendibile: ogni nuova View può avere il proprio Controller senza impattare sulle altre componenti di gioco.

Per garantire una chiara organizzazione del codice, è stato creato un controller dedicato per ogni View, ottenendo un accoppiamento forte tra View e Controller. Per la gestione centralizzata delle viste è stato introdotto un ViewManager, responsabile della creazione, visualizzazione e transizione tra le diverse View del gioco. Il ViewManager coordina il cambio di schermate, per la gestione degli input abilitati e lo stato delle viste, evitando che tali responsabilità siano distribuite nei singoli controller.

Questa soluzione rafforza la separazione delle responsabilità all'interno dell'architettura MVC e migliora la manutenibilità e la scalabilità del progetto.

Functional Requirements

- The system allows players to create and manage a party composed of two to four characters, including character selection and evolution;
- The system generates and manages enemies (foes) during exploration and combat phases;
- The system allows players to engage in combat with Destruction's minions and progress through the main storyline;
- The system allows players to make plot-related choices that influence the progression of the story;
- The system allows players to save and load game progress, including party status, learned skills, and story state;
- The system updates the game state dynamically based on Destruction's progression and the player's actions;
- The system manages NPC interactions, including dialogues, shops, and inventory management;
- The system manages boss fights against Destruction and its emanations, applying specific combat rules and behaviors;
- The system supports multiple endings, determined by the choices made by the player throughout the game.

Funzionalità implementate

Durante lo sviluppo del progetto sono state implementate le seguenti funzionalità principali:

- Avvio e struttura del progetto:
 - Definizione dell'architettura tramite diagrammi UML;
 - Implementazione del GameLauncher
- Gestione dello stato di gioco:
 - Implementazione della classe GameState, responsabile della gestione dello stato globale del progetto;
 - Separazione chiara tra logica di gioco e la rappresentazione grafica;
- Scelta dei personaggi:
 - Implementazione della selezione dei personaggi (US4);
 - Creazione del party in base alle scelte del giocatore;

- Aggiornamento dinamico della UI durante la selezione;
- Implementazione mappe:
 - Implementazione del sistema di mappe esplorabili e transizione tra mappe;
 - Introduzione del MapBuilder (US21) per la creazione modulare e riutilizzabile delle mappe, utilizzando il **Builder Pattern**;
- Esplorazione:
 - Gestione del movimento del party e delle collisioni con ambienti, nemici e NPC;
 - Caricamento corretto degli assets grafici (sprite e mappa);
- Interazione con NPC:
 - Inserimento degli NPC nelle mappe;
 - Implementazione delle interazioni base tra party e NPC;
 - Implementazione degli NPC con il sistema di dialogo e con lo shop delle abilità;
- Sistema di combattimento a turni:
 - Implementazione del sistema di combattimento a turni (US15-US16);
 - Calcolo dei danni basato sugli attributi di attacco e difesa;
 - Gestione dei turni tramite **TurnIterator**;
 - Implementazione della **BattleView**, con la visualizzazione del party, dei nemici e i turni;
 - Caricamento delle mosse da file JSON, rendendo il sistema modulare;
 - Gestione delle statistiche (HP, XP) tramite HUD dinamico;
- Sistema di abilità:
 - Implementazione della **ShopView** per l'apprendimento di nuove abilità;
 - Collegamento delle abilità ai personaggi;
 - Aggiornamento dinamico delle abilità disponibili nella UI;
- Sistema di salvataggio e caricamento:
 - Implementazione del salvataggio manuale (US9), con l'utilizzo del Memento Pattern per salvare e ripristinare lo stato del gioco;
 - Salvataggio completo del GameState;
 - Implementazione della **ContinueGameView** per il caricamento delle partite salvate;
 - Introduzione dell'**autosave**, integrato con il sistema di salvataggio manuale;
- Interazione tra i giocatori e la storia del gioco:
 - Le scelte dei giocatori influenzano attivamente la storia del gioco (US10);
 - La storia è costruita seguendo una struttura a grafo. Ogni nodo del grafo è un oggetto di tipo *StoryNode*, che rappresenta uno specifico punto della trama del gioco e propone da 1 a 2 scelte che portano a un nodo diverso del grafo;
 - Implementazione della **StoryView** che mostra a schermo i dialoghi e le scelte possibili del rispettivo **StoryController**

- Boss finale e conclusione del gioco:
 - Implementazione del Boss con statistiche e abilità uniche;
 - Integrazione del Boss nel sistema di combattimento;
 - Preparazione della logica per il finale della storia;
- Test e stabilizzazione:
 - Esecuzione di test sulle principali classi del progetto;
 - Test di integrazione tra i vari componenti;
 - Gestione dei casi limite;
 - Stabilizzazione generale del gioco;

1. Avvio e flusso di gioco (Game Lifecycle)

- **US1** – Game launch.

Il GameLauncher è implementato tramite il Singleton Pattern, così da garantire l'esistenza di un'unica istanza responsabile dell'inizializzazione del gioco, fornendo un punto di accesso globale controllato. Si è scelto di utilizzare tale pattern per evitare conflitti durante la fase di avvio e centralizzare la gestione dello stato iniziale.

- **US3/US20** – Start game/Continue Game

Per l'inizio della partita si è scelto di chiedere all'utente tramite View se iniziare una nuova partita da capo o se continuarne una già esistente. La logica della scelta della View da mostrare risiede nei Controller. In particolare, poiché le opzioni di scelta degli utenti sono molto limitate, si è scelto di applicare il design pattern **Factory** per delegare la creazione di un oggetto di tipo *GameController* per nascondere al *MainMenuController* i dettagli dell'istanziamento di un oggetto di tale classe.

La User Story Continue Game è stata accorpata alla Start Game, in quanto entrambe rappresentano modalità alternative di avvio del gioco e condividono lo stesso flusso iniziale.

- **US7/US8** – Early ending & End of story termination

Le User Story 7 e 8, End of story termination ed Early ending, non sono implementate in maniera esplicita, ma consistono nelle azioni dell'utente di mettere in pausa il gioco, durante la partita o dopo la battaglia con il boss finale, scegliere se salvare e andare alla schermata principale

2. Salvataggi e caricamento (Save / Load)

- **US2/US9** – Manual save into slot & Autosave

Il pattern che si è deciso di utilizzare è il Memento Pattern. Esso ci permette di salvare e ripristinare lo stato del gioco senza violare l'incapsulamento.

Struttura: il GameState rappresenta l'**Originator**, il GameStateMemento rappresenta lo Snapshot e la classe SaveManager è il **Caretaker** cioè, gestisce gli slot di salvataggio e si limita di conservare il Memento e richiedere il ripristino. L'auto-save è stato integrato nel sistema di salvataggio manuale attraverso dei metodi aggiuntivi alle classi menzionate.

- **US5** – Select save slot and continue.

Questa User Story viene implementata tramite la creazione della ContinueGameView, una UI dedicata alla selezione dello slot di salvataggio per continuare il gioco da uno dei salvataggi effettuati.

- **US6** – Notification for corrupted save file.

Per soddisfare la User Story è stata implementata una procedura di validazione dei file di salvataggio che consente di rilevare la presenza di un file corrotti e di notificare correttamente l'utente. Il metodo isSlotValid(int slot), implementato nella classe SaveManager, verifica l'esistenza del file di salvataggio associato allo slot selezionato. In assenza del file, lo slot viene considerato non valido. Nel caso in cui il file esista, il sistema tenta di deserializzarlo utilizzando la libreria Jackson, convertendolo in un oggetto di tipo GameStateMemento. Se l'operazione va a buon fine, il file viene considerato valido e utilizzabile per il caricamento della partita.

Se la deserializzazione genera un'eccezione di tipo IOException, il file viene indicato come corrotto. Tale approccio garantisce una maggiore robustezza del sistema di salvataggio e migliora l'esperienza utente fornendo i feedback necessari sul caricamento degli slot di salvataggio. La notifica avviene mediante la classe ViewManager

```
public void showCorruptSave(int slot) {
    Alert alert = new Alert(Alert.AlertType.ERROR);
    alert.setTitle("Error while loading the save");
    alert.setHeaderText("Slot " + slot + " contains a corrupt
save file");
    alert.showAndWait();
}

public void showCorruptSave() {
    Alert alert = new Alert(Alert.AlertType.ERROR);
```

```
alert.setTitle("Error while loading the save");
alert.setHeaderText("Autosave slot is corrupted");
alert.showAndWait();
}
```

3. Personaggio e progressione (Character Progression)

- **US4** – Choose character job

La user story viene implementata usando l'Abstract Factory per gestire la creazione dei personaggi di gioco in modo flessibile e disaccoppiato dal controllo principale dell'applicazione. In base alla classe Enum Job, il Sistema crea una specifica factory per istanziare oggetti Player ed Enemy coerenti tra loro in termini di caratteristiche e comportamento. Questo approccio consente di garantire la consistenza logica tra i personaggi generati, evitando accoppiamenti diretti tra il GameController e le classi concrete di implementazione.

Il GameController interagisce esclusivamente con le interfacce astratte fornite dalla factory, delegando completamente la responsabilità di creazione degli oggetti alle implementazioni concrete. In questo modo, il sistema risulta facilmente estendibile: l'aggiunta di una nuova classe di personaggio richiede esclusivamente la creazione di una nuova factory concreta, senza modificare la logica di controllo del gioco.

- **US17-US18** – Select new ability & Character evolution

Le User Story 17 e 18 è implementata all'interno dello stesso contesto architetturale della US Combat System, riutilizzando il pattern Strategy. L'evoluzione del personaggio avviene tramite l'introduzione del metodo **learnMove(Move move)** nella classe player, che consente di apprendere dinamicamente nuove mosse durante il gioco. Le nuove mosse vengono integrate nell'insieme delle strategie disponibili, senza richiedere modifiche alle classi esistenti.

```
public void learnMove(Move move) {
    if (!this.getCurrentMove().contains(move)) {
        this.getCurrentMove().add(move);
    }
}
```

- **US19** – Update character data after learning skill.

Quando il personaggio apprende una nuova abilità tramite il metodo `learnMove(Move move)`, la nuova mossa viene aggiunta dinamicamente all'insieme delle strategie disponibili del Player. Tale operazione comporterà una modifica dello stato interno del model, in particolare dell'elenco delle abilità utilizzabili. il Pattern Observer garantirà che ogni cambiamento dello stato del Player venga automaticamente notificato alla view, così da aggiornare in tempo reale la UI. Questo approccio favorisce la coerenza dei dati visualizzati e assicura che l'evoluzione del personaggio sia immediatamente visibile all'utente.

4. Combattimento (Combat System)

- **US16** – Turn based battles

Si è scelto di implementare delle battaglie a turni che si svolgono nel seguente modo: tutti i giocatori scelgono un'Action da eseguire e specificano il bersaglio. Il sistema ordina tali Action utilizzando un certo criterio di ordinamento, che viene passato alla classe *Battle* come attributo di tipo *TurnStrategy*. Un oggetto che ha il riferimento a *TurnStrategy* è un oggetto che implementa l'interfaccia *TurnStrategy* così come definita:

```
public interface TurnStrategy {  
    public abstract void sortAction();  
    public abstract TurnIterator getTurnIterator();  
}
```

Si usa infatti il design pattern **Strategy**, cosicché la classe *Battle* possa effettuare l'ordinamento senza conoscere la relazione d'ordine implementata. In questo modo possiamo cambiare le regole della battaglia o creare specifiche battaglie personalizzate creando una nuova classe che implementa l'interfaccia *TurnStrategy*.

Internamente le classi che implementano *TurnStrategy* utilizzano internamente una struttura dati (noi abbiamo scelto una lista). Per questo motivo si è usato un **Iterator** per poter delegare a una classe esterna a *Battle* l'esplorazione della struttura dati.

Le Action possibili all'interno del gioco consistono nella scelta di una Move da eseguire su un bersaglio. La classe Action ha quindi come attributo quattro oggetti: un *CharacterPG* (l'utilizzatore), una *List<CharacterPG>* (i bersagli, che eventualmente possono essere soltanto uno), l'*ActionStrategy* da applicare e una *String* che contiene un riferimento testuale univoco a una

Move. Questi ultimi 2 oggetti sono mutualmente esclusivi, nel senso che non intervengono mai contemporaneamente durante la risoluzione di un' *Action*.

```
public void execute() {
    if (action2 != null) {
        action2.doAction(user, targets);
    } else if (action != null) {
        MoveRegistry.getMoveRegistry().
            getMoveActionStrategy(action).
            doAction(user, targets);
    }
}
```

Per capire il funzionamento delle *Move* dal punto di vista tecnico ci si sofferma sul secondo blocco di questo snippet di codice.

MoveRegistry è un **Singleton**, infatti il metodo static *getMoveRegistry()* restituisce l'istanza del *MoveRegistry* attualmente attiva (è un oggetto static) oppure, se l'istanza è null, inizializza il *MoveRegistry*.

```
private MoveRegistry() {
    this.moves= MoveReader.readMove("/battle/moves.json")
        .stream()
        .collect(Collectors.toMap(Move::getName, m->m));
}

public static MoveRegistry getMoveRegistry() {
    if (moveRegistry == null) {
        moveRegistry= new MoveRegistry();
    }
    return moveRegistry;
}
```

All'interno del costruttore, dichiarato come **private** perché l'unico accesso al **Singleton** è dato dall'opportuno metodo static, si fa riferimento al metodo static *readMove(String path)* della classe *MoveReader*. Si è infatti scelto di usare il design pattern **Adapter** per convertire le mosse dalla risorsa testuale "moves.json" nel dato *Move*. In questo caso *MoveReader* è infatti l'adapter che converte la rappresentazione json in una rappresentazione di tipo *List<Move>* utilizzabile da *MoveRegistry*.

Questa scelta, in combinazione con l'utilizzo delle *ActionStrategy*, permette di astrarre completamente il comportamento di una *Move* e di modificarlo semplicemente modificando il file di testo "moves.json", così come di aggiungere nuove *Move*.

- **US14** – Fighting enemies.

La user story è implementata mediante il pattern Observer, in quanto lo stato della battaglia varia dinamicamente e la view deve essere aggiornata automaticamente in seguito a tali cambiamenti, senza introdurre dipendenze dirette tra modello e interfaccia grafica. L'utilizzo di tale pattern, consente di mantenere il sistema disaccoppiato, separando la logica di gioco dalla rappresentazione grafica.

Il model Battle notifica le viste ogni volta che si verifica una modifica significativa dello stato del combattimento, come variazioni dei punti vita e sconfitta di un personaggio. In questo modo, la view può reagire agli aggiornamenti senza mantenere riferimenti diretti alle classi del modello.

- **US15** – Damage, effect & healing calculation

La gestione degli effetti delle *Move* è delegata all'interfaccia *ActionStrategy*. Si fa uso del design pattern **Strategy**: si dichiara l'interfaccia *ActionStrategy* che viene implementata da *MoveAction*, che è una classe astratta che fa da radice a una gerarchia di classi che implementano la stessa interfaccia.

```
public interface ActionStrategy {  
    public void doAction(CharacterPG user,  
                        List<? extends CharacterPG> target);  
}
```

In questo modo è possibile dare effetti e modi di risolvere diversi a ogni tipo di mossa a condizione che si trovi tutto nei metodi dichiarati da *ActionStrategy*. Le classi che implementano l'interfaccia, comunque, usano dei metodi **private** interni come helper.

In particolare, si ha la seguente gerarchia di *MoveAction*.

```
MoveAction* -> OffensiveMove* -> AttackMove  
           -> MagicAttackMove  
           -> HealingMove
```

*Le classi contrassegnate con * sono classi astratte che aiutano a impostare la gerarchia.

La classe astratta *MoveAction* ha come attributo un oggetto di tipo *Move* da cui prendere tutti i valori necessari per il calcolo dei danni o cura.

5. Esplorazione e mondo di gioco (World & Exploration)

- **US13** – Exploration movement

Il movimento del party nella mappa viene implementato usando il pattern Observer, in modo che il controller, attraverso un metodo appropriato nella classe Party, modifichi la posizione solo del giocatore principale, che si occuperà di notificare il follower che, a catena, notificherà e farà spostare tutti i membri del party.

```
public class Party {  
    private final List<Player> members = new ArrayList<>();  
  
    public void updateFollowPosition(Position newLeaderPos) {  
        getMainPlayer().notifyFollower();  
        getMainPlayer().setPosition(newLeaderPos);  
    }  
}
```

```
public void notifyFollower() {  
  
    if (this.follower != null) {  
  
        this.follower.notifyFollower();  
  
        this.follower.setPosition(getPosition());  
  
    }  
  
}
```

- **US21** – MapView

Essendo la MapView un oggetto molto complesso da istanziare, con molti attributi che devono essere impostati nel costruttore, viene utilizzato il pattern Builder, implementato attraverso la classe MapBuilder, che viene chiamato all'interno dell'Exploration View

```
TiledMapData mapData = MapBuilder  
    .loadRawMapData(map.getFilePath());  
this.mapView = new MapBuilder()  
    .setDimensions(  
        mapData.getTileheight(),  
        mapData.getWidth(),
```

```

        mapData.getHeight() )
        .setWalkableIds (TILESET_DATA_PATH)
        .setTileSetImageFromPath (TILESET_IMAGE_PATH)
        .addLayers (mapData.getLayers() )
        .showSprites (map.getSpriteindex() )
        .build();

```

6. Interazioni e narrativa (Story & Interactions)

- **US10** – Story choices influence future

Si è soddisfatto il requisito funzionale che richiede che le scelte effettuate dai giocatori influenzino attivamente la trama del gioco. Per farlo abbiamo progettato la trama del gioco sin dall'inizio ed elaborato un grafo che possa rappresentare ogni punto della storia.

Ogni nodo del grafo è un oggetto di tipo *StoryNode* che contiene un identificativo testuale, una *List<String>* contenente i dialoghi che lo *StoryController* farà visualizzare a schermo dalla *StoryView*, una *List<Choice>*, contenente le scelte che possono essere compiute dai giocatori e una funzione di tipo *SerializablePredicate<GameState>** (chiamato trigger) che testa il *GameState* per stabilire se è opportuno risolvere lo *StoryNode* corrente.

```

public interface SerializablePredicate<T>
    extends Predicate<T>, Serializable {}

```

Costruire un oggetto di tipo *StoryNode* è complesso: bisogna istanziare singolarmente ogni *Choice*, inoltre non sempre si vuole usare uno specifico trigger per gli *StoryNode*. Per questo motivo si è scelto di usare il design pattern **Builder**: Si costruisce lo *StoryNode* passo dopo passo con un oggetto di tipo *StoryNodeBuilder*. Di default il trigger dello *StoryNodeBuilder* è una funzione che restituisce sempre il boolean true. Inoltre il builder permette di aggiungere singolarmente ogni dialogo (risp. *Choice*) o impostare direttamente una *List<String>* (risp. *List<Choice>*).

Un oggetto di tipo *Choice* contiene invece un riferimento testuale, non necessariamente univoco, un riferimento allo *StoryNode* a cui quella *Choice* conduce e una funzione di tipo *SerializableConsumer<GameState>** (detta consumer)

```

public interface SerializableConsumer<T>
    extends Consumer<T>, Serializable {}

```

Si è scelto di usare l'**Adapter** *StoryNodeLoader* per trasformare la rappresentazione testuale degli *StoryNode* ("dialogues.json") in una rappresentazione utilizzabile dal sistema. Si nota che *StoryNodeLoader* non restituisce una *List<StoryNode>*, ma un solo *StoryNode*, che è la radice del grafo.

*L'utilizzo delle interfacce del package *java.util.function* è dovuto all'idea di voler passare come parametro delle espressioni lambda, tuttavia abbiamo avuto problemi tecnici nel rappresentarle testualmente in formato JSON. Da qui la necessità di scrivere delle classi vere e proprie che implementano le interfacce *SerializablePredicate<T>* e *SerializableConsumer<T>*.

- **US12** – NPC interactions.

Il metodo *handlePossibleInteractions()* gestisce le interazioni tra il *mainPlayer* e gli NPC presenti nel gioco. La funzione verifica se un NPC è sufficientemente vicino al giocatore, valutando la distanza tra le loro posizioni tramite la differenza massima degli assi. Per ogni NPC presenti nella lista presente nel *GameState*, viene calcolata la distanza massima assoluta lungo le coordinate rispetto alla posizione del giocatore principale. Se tale distanza è minore o uguale a 1, l'NPC viene selezionato come bersaglio per l'interazione. Se un NPC target è stato identificato, il metodo richiama *ViewManager* per mostrare la finestra di dialogo con l'NPC, passando alla scena corrente, il giocatore principale e l'NPC selezionato. Successivamente, viene invocato il metodo *interact()* sull'NPC, fornendo il party come contesto, per attivare la logica di interazione specifica.

```
public void handlePossibleInteractions() {
    NPC target = null;
    Party party = GameState.getInstance().getParty();
    Player mainPlayer = party.getMainPlayer();
    for (NPC npc : GameState.getInstance().getNpc()) {
        if (Math.abs(npc.getPosition()
            .sub(mainPlayer.getPosition()).max()) <= 1) {
            target = npc;
            break;
        }
    }

    if (target != null) {
        ViewManager.getInstance()
            .showDialogView(scene, mainPlayer, target);
        target.interact(party);
    }
}
```

}

Conclusioni

Il progetto ha permesso di sviluppare un videogioco seguendo un approccio agile, basato su User Stories che hanno guidato l'implementazione delle funzionalità principali. L'utilizzo dei design pattern, in particolare **Abstract Factory**, **Strategy**, **Observer** e **Memento**, ha garantito un'architettura modulare, scalabile e facilmente estendibile, consentendo di separare la logica di gioco dalla gestione della UI e dal controllo delle interazioni tra i personaggi.

La progettazione orientata ai pattern ha reso possibile l'implementazione di funzionalità complesse, come la gestione dinamica delle mosse, l'evoluzione del personaggio ed interazione con NPC e nemici, in modo riusabile e disaccoppiato, senza introdurre dipendenze dirette tra componenti critiche del sistema. Tale approccio ha inoltre facilitato la manutenzione del codice e la tracciabilità dei requisiti, garantendo che ogni User Story fosse implementata in maniera coerente e verificabile.

In futuro, l'architettura adottata permette di estendere il gioco con nuove classi di personaggi, abilità, nemici o sistemi di interazione aggiuntivi, senza richiedere modifiche invasive alle componenti esistenti, confermando con robustezza e la flessibilità del design.